

ANÁLISIS Y DISEÑO DE LA PRUEBA

Cap 4 | Fundamentos de pruebas de software

4

Las **pruebas estáticas** y **dinámicas** se complementan entre sí para hacer pruebas efectivas. Las revisiones y análisis estático encuentran defectos, mientras que las pruebas dinámicas encuentran fallas. Las fallas solo ocurren cuando el código es ejecutado.

ELECCIÓN DE TÉCNICAS DE PRUEBAS

Algunas técnicas formales o informales son más adecuadas para ciertas situaciones y niveles de prueba; otras se pueden aplicar en todos los niveles de prueba. Pretender que una sola técnica de prueba te proporcionará los resultados que esperas para la totalidad de un proyecto, suele ser una postura optimista, en la mayoría de los casos será necesario implementar una combinación de técnicas para lograr abarcar los casos que deseamos.

TÉCNICAS DE PRUEBAS

Las **técnicas de caja negra** o basadas en especificaciones son apropiadas en todos los niveles de prueba donde exista una especificación, desde la prueba de componentes hasta la prueba de aceptación.

Las **técnicas de caja blanca** o basadas en la estructura, también se pueden utilizar en todos los niveles de prueba. Los desarrolladores utilizan técnicas basadas en estructuras en las pruebas de componentes y las pruebas de integración de componentes, especialmente cuando hay un buen soporte de herramientas para la cobertura de código.

Las **técnicas basadas en la experiencia** se utilizan para complementar las técnicas de caja negra y blanca, y cuando la especificación es inadecuada, no existe, o no está actualizada. Este puede ser el único tipo de técnica utilizada para sistemas de bajo riesgo, y ser particularmente útil bajo una presión de tiempo extrema.



Las pruebas dinámicas constan de tres técnicas principales: Caja blanca, Caja negra y Basadas en la experiencia

TÉCNICAS DE PRUEBA DE CAJA NEGRA

Las técnicas de caja negra ven el software como un recuadro negro con entradas y salidas, pero no tienen conocimiento de cómo funciona el sistema. El probador se concentra en lo que hace el software, no en cómo lo hace. Las técnicas de caja negra se pueden listar:

- Partición de equivalencia

También se le conoce como **clases de equivalencia**, divide los datos en particiones basándose en la expectativa de todos los elementos de una partición determinada.

Si un caso que prueba un valor de una partición detecta un defecto, también será detectado para cualquier otro valor dentro de la partición, por lo que requiere un caso para cada una de ellas.

Las particiones pueden tener valores válidos o inválidos, según sea necesario según el objetivo perseguido.

Sirva el siguiente escenario como ejemplo:

En un sistema electoral tenemos la variable “Edad para votar”, hay personas mayores de edad y menores de edad, todas las personas dentro de cada grupo sin importar los años que tengan, serán tratados de la misma forma en un proceso electoral.

Menores de Edad

Mayores de Edad

$0 < \text{Menor} < 18$

$18 \leq \text{Mayor}$

Valor a probar: 15

Valor a probar: 21



“Cada elección” es un criterio de cobertura utilizado por ejemplo para objetos de prueba con múltiples particiones (ej. Cuando hay más de un parámetro de entrada). Exige que se utilice cada partición para cada conjunto de particiones al menos una vez, pero no tiene en cuenta combinaciones de particiones.

Técnicas de Caja Negra

Partición de equivalencia

Análisis de Valores Límites

Tablas de Decisión

Transición de Estados

- Análisis del valor frontera

Consiste en tomar los valores límites de los conjuntos que nombramos anteriormente debido a que es común que, en casos de prueba relacionados con rangos, los defectos se concentren justamente en los límites de ambos conjuntos. Existen 2 técnicas al momento de implementar un “Análisis de valores Frontera”, uno está basado en el análisis de 2 puntos y otro en el análisis de 3 puntos.

Al usar la técnica de 2 puntos la Frontera hace referencia a una línea divisoria entre 2 particiones con valores a ambos lados de esa línea, pero la Frontera o el Límite en sí mismo, no es un valor.

En la técnica de 3 puntos, el valor Frontera o Límite está "EN" la Frontera, y los otros 2 valores se encuentran a los lados de dicho valor, bien sean estos provenientes de una partición válida o inválida.

Suponga un campo que acepta un valor entero único positivo entre 1 y 5. El rango válido es del 1 al 5, ambos extremos incluidos.

Sin embargo, sabemos que un campo de valor entero de un dígito puede recibir valores del 0 al 9.

Particiones de Equivalencia

Inválida (demasiado baja)	Válida	Inválida (demasiado alta)
0	1,2,3,4,5	6,7,8,9

Prueba de dos valores por frontera

Inválida - (demasiado baja)	Valida	Valida - (demasiado alta)	Inválida
0 y 1			5, 6

Prueba de tres valores por frontera

Inválida - (demasiado baja)	Valida	Valida - (demasiado alta)	Inválida
0,1 y 2			4,5 y 6

- Prueba de Tabla de Decisión

Las tablas de decisión se utilizan para probar la implementación de los requisitos de sistemas que especifican cómo diferentes combinaciones de condiciones dan lugar a diferentes resultados.

Probar todas las combinaciones puede ser poco práctico o hasta imposible, por esta razón cuando hay múltiples condiciones y salidas se pueden usar las tablas de decisión.

Para crear una tabla decisión puedes seguir los siguientes 3 pasos:

Paso 1: Identificar las condiciones y agregarlas a la tabla.

Paso 2: Identificar todas las posibles combinaciones de verdadero y falso.

Paso 3: Identificar las salidas correspondientes a cada combinación.

Veamos un ejemplo

Supongamos que tenemos una solicitud de compra de un vehículo, donde debe ingresar el modelo del vehículo y el color. Si ingresa ambos, el sistema procesa ambos datos y por consiguiente la solicitud, y en caso de que uno de los valores falte, se envía un mensaje de error.

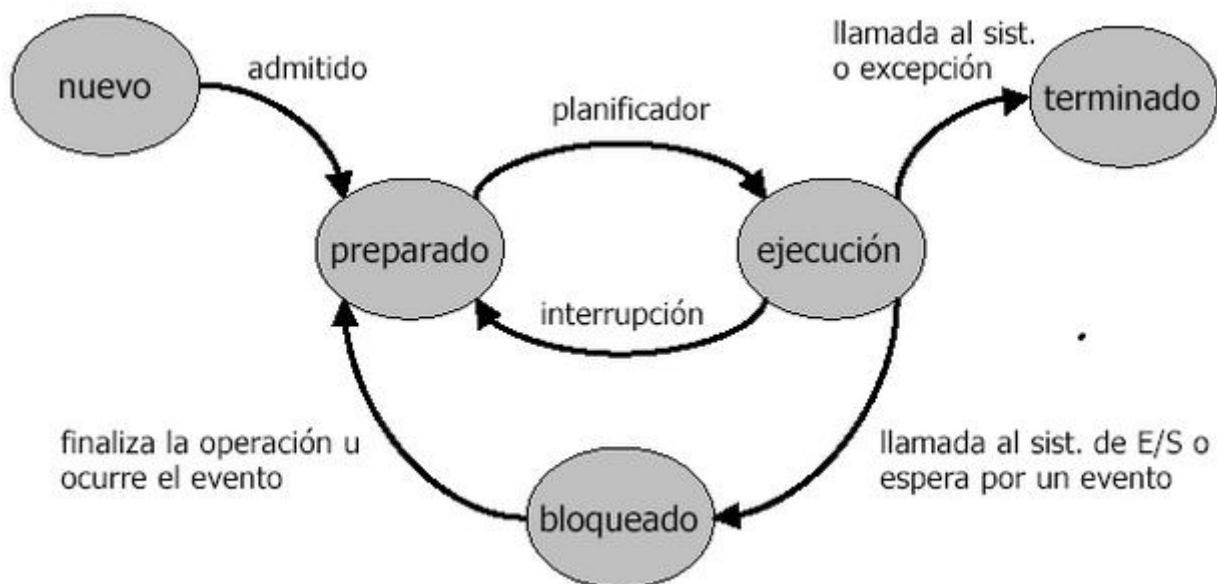
TÉCNICAS DE PRUEBA DE CAJA NEGRA

Condiciones	Combinación 1	Combinación 2	Combinación 3	Combinación 4
¿Se ingresó el modelo del vehículo?	Verdadero	Verdadero	Falso	Falso
¿Se ingresó el color del vehículo?	Verdadero	Falso	Verdadero	Falso
Acciones/Salidas				
Resultado	Se procesa la solicitud del vehículo	Mensaje de Error	Mensaje de Error	Mensaje de Error



Este método puede aplicarse a cualquier cantidad de condiciones, y garantiza que todas las posibles combinaciones han sido cubiertas, e incluso identificar redundancias o escenarios inalcanzables

- Prueba de Transición de Estado



- Pruebas de Transición de Estado

Un diagrama de transición de estados modelo el comportamiento de un sistema mostrando estados y transiciones.



El círculo representa un Estado



Las transiciones se representan con líneas con puntas para garantizar direccionalidad

Una transición es iniciada por un evento que puede ser calificado adicionalmente por una **condición de guarda**.

Existen muchos criterios de cobertura para las pruebas de transición de estado.

El syllabus considera 3

- **Cobertura de todos los estados.** 100% de cobertura implica que los casos de prueba aseguran que se pasan por todos los estados.
- **Cobertura de transiciones válidas.** 100% de cobertura implica que los casos aseguran practicar todas las transiciones válidas.
- **Cobertura de todas las transiciones.** Los elementos de cobertura son todas las transiciones que aparecen en una tabla de estados



Los Eventos y las Acciones son textos cercanos a las transiciones, se suelen separar por el símbolo de barra oblicua o slash. Primero la Acción y luego el Evento.

Condición de Guarda, está representado por Corchetes “[]”, lo que significa que si no se cumple, no se ejecutará la transición.

Encender [Si hay electricidad]

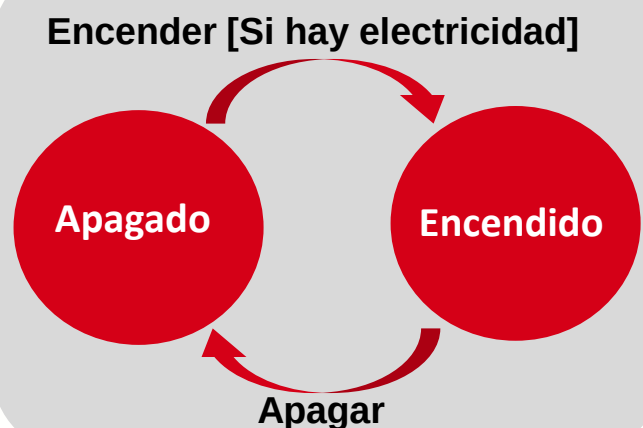


Diagrama de Transición

Las técnicas de caja blanca

o caja de cristal son llamadas así porque se puede ver que hay dentro de la caja, conocemos la estructura del código y por esa razón también se llaman pruebas basadas en la estructura.

En este grupo se estudian dos técnicas en particular: **la cobertura de sentencia y la cobertura de decisión.**



Cobertura de Sentencia

Cobertura de Decisión

Técnicas de
Caja Blanca

Cobertura de sentencia

Una sentencia es una expresión de código que puede ejecutarse como un todo, por ejemplo, un cálculo matemático, una declaración if, while o case, y la idea es cubrir cada sentencia del código fuente para validar que no existen defectos en ellas

Cobertura de decisión

Para probar las estructuras que pueden generar más de un resultado, como las condicionales, es necesario usar la técnica de cobertura de decisión para garantizar un correcto funcionamiento del código. Esto consiste en probar cada condición validando cada posible resultado verdadero o falso.

TÉCNICA DE PRUEBA BASADAS EN LA EXPERIENCIA

Algunas veces no tenemos tiempo disponible para ejecutar las pruebas antes descritas, o no hay especificaciones bien documentadas, y es allí donde entran en juego las técnicas de pruebas basadas en la experticia del probador: **predicción de errores, pruebas exploratorias y las basadas en listas de comprobación.**

Estas técnicas aprovechan la experiencia de los desarrolladores, probadores y usuarios, para diseñar, implementar y ejecutar las pruebas. Usualmente estas técnicas se combinan con las de caja negra y caja blanca.

La predicción de errores se basa en la información que se tiene de cómo ha funcionado la aplicación en el pasado, tipo de errores que tienden a cometer los desarrolladores y fallos que se han producido en otras aplicaciones.

Las pruebas exploratorias consisten en explorar el software para saber qué hace, que no hace, qué funciona y qué no, las decisiones sobre que probar se van tomando sobre la marcha.

Las pruebas basadas en listas de comprobación consisten en listar los posibles defectos/fallas que pueda presentar el software, basándose como las anteriores, en la experiencia del probador sobre por qué y cómo falla el software.



Predicción de Errores

Prueba Exploratoria

Lista de Comprobación

Técnicas
Basadas en la
Experiencia

ENFOQUES DE PRUEBA BASADOS EN LA COLABORACIÓN

Cada una de las técnicas mencionadas anteriormente tiene un objetivo particular con respecto a la detección de defectos. Los **enfoques basados en colaboración**, en cambio, **se concentran** también **en evitar defectos** mediante colaboración y comunicación.



Enfoques
basados en
Colaboración

Redacción Colaborativa de
Historias de Usuario

Criterios de Aceptación

Desarrollo Guiado por Prueba
de Aceptación

- Redacción colaborativa de Historia de Usuario (HU)

Una HU representa una prestación que será de valor para un usuario o comprador.

Tiene tres aspectos críticos (las 3 C)

- **C**ard (Cuartilla)
- **C**onversation (Conversación)
- **C**onfirmation (Confirmación)

La estructura habitual es “Como [rol], quiere que se cumpla [objetivo], de modo que pueda [valor de negocio para el rol]”

Las buenas HU deben ser

Español		Inglés
Independiente	I	I ndependent
Negociable	N	N egotiable
Valioso/a	V	V aluable
Estimable	E	E stimable
Pequeño/a (Sucinto)	S	S mall
Capacidad de Prueba	T	T estable

Criterios de Aceptación

Son condiciones que debe cumplir una implementación de HU para ser aceptada por los implicados

Uso

- Definir alcance de HU
- Alcanzar consenso
- Describir escenarios
- Servir de base para PAU
- Permitir planificar y estimar.

Su redacción puede ser

- Orientada al escenario (Given/When/Then – DGC (BDD))
- Orientada a reglas (ej. Tablas de entrada y salida)

Desarrollo Guiado por Prueba de Aceptación - Enfoque de probar primero
Se hace un taller de especificación y luego se crean casos de prueba